Vue nedir?

Vue (/vjuː/ şeklinde, İngilizce *view* kelimesi gibi okunur), kullanıcı arayüzleri oluşturmaya yarayan bir JavaScript çatısıdır (framework). Standart HTML, CSS ve JavaScript üzerine kuruludur; her tür karmaşıklıkta arayüzü verimli bir şekilde geliştirmenize yardımcı olan, deklaratif (bildirimsel) ve bileşen tabanlı bir programlama modeli sunar.

Reaktivite Temelleri (Reactivity Fundamentals)

Reaktif Durum Tanımlama (Declaring Reactive State)

Options API kullanırken, bir bileşenin reaktif durumunu tanımlamak için `data` seçeneğinden yararlanırız. Bu seçeneğin değeri, geriye bir nesne (object) döndüren bir fonksiyon olmalıdır. Vue, yeni bir bileşen örneği (instance) oluştururken bu fonksiyonu çağırır ve döndürülen nesneyi kendi reaktif sistemine dahil eder. Bu nesnenin en üst seviyedeki tüm özellikleri (properties), bileşen örneği üzerinden (metotlarda ve yaşam döngüsü kancalarında `this` ile) doğrudan erişilebilir hale getirilir.

```
export default {
  data() {
    return {
      count: 1
    }
  },

  // `mounted` is a lifecycle hook which we will explain later
  mounted() {
    // `this` refers to the component instance.
    console.log(this.count) // => 1

    // data can be mutated as well
    this.count = 2
  }
}
```

Bu özellikler (properties) yalnızca bileşen ilk oluşturulduğunda eklendiği için, `data` fonksiyonunun döndürdüğü nesne içerisinde hepsinin tanımlanmış olduğundan emin olmalısınız. Henüz hazırda bir değeri olmayan özellikler için gerektiğinde `null`, `undefined` veya başka bir geçici yer tutucu (placeholder) değer kullanabilirsiniz.

`data` içerisine dahil etmeden doğrudan `this` üzerine yeni bir özellik eklemek de mümkündür. Ancak bu şekilde eklenen özellikler reaktif güncellemeleri tetikleyemez.

Vue, kendi yerleşik API'lerini bileşen örneği üzerinden sunarken $ önekini kullanır. Dahili (internal) özellikler için ise _ önekini kendine ayırmıştır. Bu nedenle, üst seviye data özelliklerinize bu karakterlerle başlayan isimler vermekten kaçınmalısınız.

Reaktif Proxy ile Orijinal Proxy Karşılaştırması

(Reactive Proxy vs. Original)

Vue 3'te veriler, JavaScript Proxies (Vekil Nesneler) mekanizması kullanılarak reaktif hale getirilir. Vue 2'den geçiş yapan kullanıcıların aşağıdaki özel duruma (edge case) dikkat etmesi gerekir:

```
export default {
  data() {
    return {
      someObject: {}
    }
  },
  mounted() {
    const newObject = {}
    this.someObject = newObject

    console.log(newObject === this.someObject) // false
  }
}
```

Atama yaptıktan sonra `this.someObject` değerine eriştiğinizde elde edeceğiniz değer, orijinal `newObject` nesnesinin reaktif bir vekilidir (proxy). Vue 2'den farklı olarak, orijinal `newObject` nesnesi hiç değiştirilmeden olduğu gibi bırakılır ve reaktif hale getirilmez; bu nedenle reaktif duruma (state) her zaman `this` üzerinden bir özellik olarak eriştiğinizden emin olmalısınız.

Metot Tanımlama (Declaring Methods)

Bir bileşen örneğine metotlar eklemek için `methods` seçeneğini kullanırız. Bu seçenek, tanımlamak istediğiniz metotları içeren bir nesne (object) olmalıdır.

```
export default {
  data() {
    return {
      count: 0
    }
  },
  methods: {
    increment() {
      this.count++
    }
  },
  mounted() {
    // methods can be called in lifecycle hooks, or other methods!
    this.increment()
  }
}
```

Vue, metotlar için `this` bağlamını otomatik olarak bileşen örneğine (instance) bağlar. Bu sayede bir metot, bir olay dinleyicisi (event listener) veya geri çağırma fonksiyonu (callback) olarak kullanıldığında bile doğru `this` değerini korur. Metotları tanımlarken ok fonksiyonları (arrow functions) kullanmaktan kaçınmalısınız; çünkü bu kullanım Vue'nun uygun `this` değerini bağlamasını engeller.

```
export default {
  methods: {
    increment: () => {
      // BAD: no `this` access here!
    }
  }
}
```

Bileşen örneğinin diğer tüm özelliklerinde olduğu gibi, metotlara da bileşenin şablonu (template) içerisinden erişilebilir. Şablon içinde bu metotlar en yaygın olarak olay dinleyicisi (event listener) biçiminde kullanılır.

```
<button @click="increment">{{ count }}</button>
```

Yukarıdaki örnekte, `<button>` öğesine tıklandığında `increment` metodu çağrılacaktır.

Derin Reaktivite (Deep Reactivity)

Vue'da durum (state) varsayılan olarak derinlemesine reaktiftir (deeply reactive). Bu sayede iç içe geçmiş nesneleri veya dizileri doğrudan değiştirseniz (mutate etseniz) bile yapılan değişiklikler Vue tarafından algılanır.

```
export default {
  data() {
    return {
      obj: {
        nested: { count: 0 },
        arr: ['foo', 'bar']
      }
    }
  },
  methods: {
    mutateDeeply() {
      // these will work as expected.
      this.obj.nested.count++
      this.obj.arr.push('baz')
    }
  }
}
```

DOM Güncelleme Zamanlaması (DOM Update Timing)

Reaktif durumda bir değişiklik yaptığınızda DOM otomatik olarak güncellenir. Ancak bu güncellemelerin eşzamanlı (anlık/senkron) olarak uygulanmadığını unutmamak gerekir. Vue, kaç tane durum değişikliği yaparsanız yapın her bileşenin yalnızca bir kez güncellenmesini sağlamak için güncellemeleri döngüdeki bir sonraki adıma ("next tick") kadar biriktirip birlikte işler.

```
import { nextTick } from 'vue'

export default {
  methods: {
    async increment() {
      this.count++
      await nextTick()
      // Now the DOM is updated
    }
  }
}
```

Durum Bilgisi Olan Yöntemler (Stateful Methods)

Bazı durumlarda dinamik olarak bir metot fonksiyonu oluşturmamız gerekebilir; örneğin, geciktirilmiş (debounced) bir olay işleyici (event handler) tanımlarken bu yaklaşıma başvururuz:

```
import { debounce } from 'lodash-es'

export default {
  methods: {
    // Debouncing with Lodash
    click: debounce(function () {
      // ... respond to click ...
    }, 500)
  }
}
```

Ancak bu yaklaşım, yeniden kullanılan bileşenler için sorun oluşturur; çünkü geciktirilmiş (debounced) bir fonksiyon **durum bilgisi korur (stateful)**, yani geçen süreye dair dahili bir durum tutar. Birden fazla bileşen örneği aynı geciktirilmiş fonksiyonu paylaşırsa, birbirlerinin çalışmasını etkileyip çakışmaya neden olurlar.

Her bir bileşen örneğine ait geciktirilmiş fonksiyonun diğerlerinden bağımsız kalmasını sağlamak için, fonksiyonun geciktirilmiş sürümünü `created` yaşam döngüsü kancasında (lifecycle hook) oluşturabiliriz:

```
export default {
  created() {
    // each instance now has its own copy of debounced handler
    this.debouncedClick = debounce(this.click, 500)
  },
  unmounted() {
    // also a good idea to cancel the timer
    // when the component is removed
    this.debouncedClick.cancel()
  },
  methods: {
    click() {
      // ... respond to click ...
    }
  }
}
```

Hesaplanmış Özellikler (Computed Properties)

Temel Örnek (Basic Example)

Şablon içi (in-template) ifadeler oldukça kullanışlıdır; ancak bunlar yalnızca basit işlemler için tasarlanmıştır. Şablonlarınıza çok fazla mantıksal işlem (logic) yüklemek, onları karmaşıklaştırıp bakımı zor hale getirebilir. Örneğin, iç içe geçmiş bir dizi içeren şöyle bir nesnemiz olduğunu varsayalım:

```
export default {
  data() {
    return {
      author: {
        name: 'John Doe',
        books: [
          'Vue 2 - Advanced Guide',
          'Vue 3 - Basic Guide',
          'Vue 4 - The Mystery'
        ]
      }
    }
  }
}
```

Ve yazarın zaten yayınlanmış kitapları olup olmadığına bağlı olarak farklı mesajlar göstermek istediğimizi düşünelim:

```
<p>Has published books:</p>
<span>{{ author.books.length > 0 ? 'Yes' : 'No' }}</span>
```

Bu noktada şablon biraz kalabalıklaşmaya ve karmaşık görünmeye başlar. Yapılan işlemin `author.books` verisine bağlı bir hesaplama olduğunu anlamak için şablona şöyle bir alıcı gözle bakmak gerekir. Daha da önemlisi, bu hesaplamayı şablon içinde birden fazla yerde kullanmamız gerekirse aynı kodları tekrar yazmak istemeyiz.

İşte bu yüzden, reaktif veriler içeren karmaşık mantıksal işlemler için **hesaplanmış özellik (computed property)** kullanılması önerilir. Aynı örneğin refactor edilmiş (yeniden düzenlenmiş) hali şu şekildedir:

```
export default {
  data() {
    return {
      author: {
        name: 'John Doe',
        books: [
          'Vue 2 - Advanced Guide',
          'Vue 3 - Basic Guide',
          'Vue 4 - The Mystery'
        ]
      }
    }
  },
  computed: {
    // a computed getter
    publishedBooksMessage() {
      // `this` points to the component instance
      return this.author.books.length > 0 ? 'Yes' : 'No'
    }
  }
}
```

```
<p>Has published books:</p>
<span>{{ publishedBooksMessage }}</span>
```

Burada `publishedBooksMessage` adında bir hesaplanmış özellik (computed property) tanımladık.

Uygulama verisindeki `books` dizisinin değerini değiştirmeyi denerseniz, `publishedBooksMessage` özelliğinin de buna paralel olarak nasıl güncellendiğini görebilirsiniz.

Hesaplanmış özelliklere, tıpkı normal bir veri özelliği gibi şablonlar içinden veri bağlama (data-binding) uygulayabilirsiniz. Vue, `this.publishedBooksMessage` özelliğinin `this.author.books` verisine bağımlı olduğunu bilir; dolayısıyla `this.author.books` değiştiğinde, `this.publishedBooksMessage` özelliğine bağlı olan tüm alanları otomatik olarak günceller.

Şablon içindeki ifadede bir metot çağırarak da aynı sonucu elde edebileceğimizi fark etmiş olabilirsiniz:

Hesaplanmış Özellik Önbelleklemesi ve Metotlar (Computed Caching vs. Methods)

```
<p>{{ calculateBooksMessage() }}</p>
```

```
// in component
methods: {
  calculateBooksMessage() {
    return this.author.books.length > 0 ? 'Yes' : 'No'
  }
}
```

Aynı fonksiyonu hesaplanmış bir özellik (computed property) yerine metot olarak da tanımlayabiliriz. Elde edilen nihai sonuç açısından iki yaklaşım da birebir aynıdır.

Aralarındaki temel fark ise **hesaplanmış özelliklerin reaktif bağımlılıklarına göre önbelleğe alınmasıdır (cached)**. Hesaplanmış bir özellik, yalnızca reaktif bağımlılıklarından biri değiştiğinde yeniden hesaplanır. Bu da `author.books` verisi değişmediği sürece, `publishedBooksMessage` özelliğine kaç kez erişilirse erişilsin, alıcı (getter) fonksiyonu tekrar çalıştırmadan doğrudan daha önce hesaplanan sonucu döndüreceği anlamına gelir.

Bu durum aynı zamanda aşağıdaki hesaplanmış özelliğin hiçbir zaman güncellenmeyeceği anlamına gelir; çünkü `Date.now()` reaktif bir bağımlılık değildir:

```
computed: {
  now() {
    return Date.now()
  }
}
```

Buna karşın bir metot çağrısı, bileşen ne zaman yeniden işlense (re-render edilse) fonksiyonu her defasında baştan çalıştırır.

Peki önbelleğe almaya (caching) neden ihtiyaç duyarız? Örneğin elimizde büyük bir diziyi döngüye sokup üzerinde ağır hesaplamalar yapmayı gerektiren, maliyetli bir `list` hesaplanmış özelliği (computed property) olduğunu düşünelim. Dahası, bu `list` özelliğine bağımlı çalışan başka hesaplanmış özelliklerimiz de bulunabilir.

Önbelleğe alma mekanizması olmasaydı, `list` özelliğinin alıcı (getter) fonksiyonunu ihtiyaç duyulandan katbekat fazla çalıştırmak zorunda kalırdık! Önbelleğe almayı **istemediğiniz** durumlarda ise hesaplanmış özellik yerine metot çağrısı kullanabilirsiniz.

Yazılabilir Hesaplanmış Özellikler (Writable Computed)

Hesaplanmış özellikler (computed properties) varsayılan olarak yalnızca salt okunurdur (getter-only). Hesaplanmış bir özelliğe yeni bir değer atamaya çalışırsanız çalışma zamanı (runtime) uyarısı alırsınız. "Yazılabilir" (writable) bir hesaplanmış özelliğe ihtiyaç duyduğunuz nadir durumlarda ise hem bir alıcı (getter) hem de bir yazıcı (setter) tanımlayarak bunu oluşturabilirsiniz:

```
export default {
  data() {
    return {
      firstName: 'John',
      lastName: 'Doe'
    }
  },
  computed: {
    fullName: {
      // getter
      get() {
        return this.firstName + ' ' + this.lastName
      },
      // setter
      set(newValue) {
        // Note: we are using destructuring assignment syntax here.
        [this.firstName, this.lastName] = newValue.split(' ')
      }
    }
  }
}
```

Artık `this.fullName = 'John Doe'` kodunu çalıştırdığınızda yazıcı (setter) fonksiyon tetiklenecek; böylece `this.firstName` ve `this.lastName` değerleri buna uygun şekilde güncellenecektir.

Önceki Değeri Alma (Getting the Previous Value)

- Yalnızca 3.4 ve üzeri sürümlerde desteklenir

İhtiyaç duymanız halinde, alıcı (getter) fonksiyonunun ikinci parametresine erişerek hesaplanmış özelliğin döndürdüğü **bir önceki değere** ulaşabilirsiniz:

```
export default {
  data() {
    return {
      count: 2
    }
  },
  computed: {
    // This computed will return the value of count when it's less or equal to
3.
    // When count is >=4, the last value that fulfilled our condition will be
returned
    // instead until count is less or equal to 3
    alwaysSmall(_, previous) {
      if (this.count <= 3) {
        return this.count
      }

      return previous
    }
  }
}
```

Yazılabilir (writable) bir hesaplanmış özellik kullanıyorsanız:

```
export default {
  data() {
    return {
      count: 2
    }
  },
  computed: {
```

```
    alwaysSmall: {
      get(_, previous) {
        if (this.count <= 3) {
          return this.count
        }

        return previous;
      },
      set(newValue) {
        this.count = newValue * 2
      }
    }
  }
}
```

En İyi Uygulamalar (Best Practices)

Getter'lar Yan Etkisiz Olmalıdır (Getters should be side-effect free)

Hesaplanmış alıcı (computed getter) fonksiyonlarının yalnızca saf hesaplama (pure computation) yapması ve herhangi bir yan etkiden (side effect) arındırılmış olması son derece önemlidir. Örneğin, hesaplanmış bir getter içinde başka durumları (state) değiştirmeyin, asenkron isteklerde bulunmayın veya DOM üzerinde değişiklik yapmayın!

Hesaplanmış bir özelliği, bir değerin diğer değerlere dayanarak nasıl türetildiğini bildirisel (declarative) olarak ifade eden bir yapı gibi düşünün; tek sorumluluğu o değeri hesaplamak ve döndürmek olmalıdır.

Hesaplanmış Değeri Değiştirmekten Kaçının (Avoid mutating computed value)

Hesaplanmış bir özellikten (computed property) dönen değer, türetilmiş bir durumdur (derived state). Bunu anlık bir görüntü (snapshot) gibi düşünebilirsiniz; kaynak durum her değiştiğinde yeni bir anlık görüntü oluşturulur.

Anlık bir görüntüyü doğrudan değiştirmeye çalışmak mantıklı değildir; bu nedenle hesaplanmış bir dönüş değeri **salt okunur (read-only)** olarak ele alınmalı ve kesinlikle

doğrudan değiştirilmemelidir. Bunun yerine, yeni hesaplamaları tetiklemek için bağlı olduğu kaynak durumu güncellemelisiniz.

Sınıf ve Stil Bağlamaları (Class and Style Bindings)

Veri bağlama (data binding) işlemlerinde en sık karşılaşılan ihtiyaçlardan biri, bir öğenin sınıf listesini (class list) ve satır içi stillerini (inline styles) dinamik olarak yönetmektir. `class` ve `style` birer nitelik (attribute) olduğundan, diğer niteliklerde olduğu gibi `v-bind` kullanarak bunlara dinamik olarak dize (string) değerler atayabiliriz.

Ancak bu değerleri dize birleştirme (string concatenation) yöntemleriyle oluşturmaya çalışmak zahmetli ve hataya açık olabilir. Bu nedenle Vue, `v-bind` direktifi `class` ve `style` ile kullanıldığında özel kolaylıklar sunar. İfadeler, dizelerin yanı sıra nesne (object) veya dizi (array) türünde değerler de alabilir.

HTML Sınıflarını Bağlama (Binding HTML Classes)

Nesnelere Bağlama (Binding to Objects)

Sınıfları dinamik olarak açıp kapatmak için `:class` (`v-bind:class` kısaltması) ifadesine bir nesne (object) geçirebiliriz:

```
<div :class="{ active: isActive }"></div>
```

Yukarıdaki sözdizimi (syntax), `active` sınıfının varlığının `isActive` veri özelliğinin doğruluk değerine (truthiness) bağlı olarak belirleneceği anlamına gelir.

Nesne içinde daha fazla alan tanımlayarak aynı anda birden fazla sınıfı açıp kapatabilirsiniz. Buna ek olarak `:class` direktifi, normal `class` niteliğiyle de birlikte (yan yana) kullanılabilir. Dolayısıyla, aşağıdaki duruma (state) sahip olduğumuzu varsayarsak:

```
data() {
  return {
    isActive: true,
    hasError: false
  }
}
```

Ve şu şablon:

```
<div
  class="static"
  :class="{ active: isActive, 'text-danger': hasError }"
></div>
```

Şu şekilde görüntülenecektir:

```
<div class="static active"></div>
```

`isActive` veya `hasError` değiştiğinde sınıf listesi de buna bağlı olarak güncellenecektir. Örneğin, `hasError` değeri `true` olduğunda sınıf listesi "static active text-danger" şeklini alacaktır.

Bağlanan nesnenin satır içi (inline) olması gerekmez:

```
data() {
  return {
    classObject: {
      active: true,
      'text-danger': false
    }
  }
}
```

```
<div :class="classObject"></div>
```

Bu kod şu çıktıyı üretecektir:

```
<div class="active"></div>
```

Bir nesne döndüren hesaplanmış bir özelliğe (computed property) de bağlama yapabiliriz. Bu, son derece yaygın ve güçlü bir desendir (pattern):

```
data() {
  return {
    isActive: true,
    error: null
```

```
  }
},
computed: {
  classObject() {
    return {
      active: this.isActive && !this.error,
      'text-danger': this.error && this.error.type === 'fatal'
    }
  }
}
```

```
<div :class="classObject"></div>
```

Dizilere Bağlama (Binding to Arrays)

Bir sınıf listesi uygulamak için `:class` direktifini bir diziye (array) bağlayabiliriz:

```
data() {
  return {
    activeClass: 'active',
    errorClass: 'text-danger'
  }
}
```

```
<div :class="[activeClass, errorClass]"></div>
```

Bu kod şu çıktıyı üretecektir:

```
<div class="active text-danger"></div>
```

Listedeki bir sınıfı aynı zamanda koşullu olarak açıp kapatmak istiyorsanız bunu bir üçlü koşul ifadesiyle (ternary expression) yapabilirsiniz:

```
<div :class="[isActive ? activeClass : '', errorClass]"></div>
```

Bu kod her durumda `errorClass` sınıfını uygulayacaktır, ancak `activeClass` yalnızca `isActive` değeri doğru (truthy) olduğunda eklenecektir.

Ancak birden fazla koşullu sınıfınız varsa bu kullanım biraz uzun ve karmaşık gelebilir. Bu nedenle, dizi sözdiziminin (array syntax) içinde nesne sözdizimini (object syntax) kullanmak da mümkündür:

```
<div :class="[{ [activeClass]: isActive }, errorClass]"></div>
```

Bileşenlerle Birlikte (With Components)

Tek bir kök öğeye (root element) sahip bir bileşen (component) üzerinde `class` niteliği kullandığınızda, bu sınıflar bileşenin kök öğesine eklenir ve halihazırda üzerinde bulunan mevcut sınıflarla birleştirilir.

Örneğin, şu şablona (template) sahip `MyComponent` adında bir bileşenimiz olduğunu varsayalım:

```
<!-- child component template -->
<p class="foo bar">Hi!</p>
```

Ardından, bu bileşeni kullanırken bazı sınıflar ekleyelim:

```
<!-- when using the component -->
<MyComponent class="baz boo" />
```

Oluşturulan HTML çıktısı şu şekilde olacaktır:

```
<p class="foo bar baz boo">Hi!</p>
```

Aynı durum sınıf bağlamaları (`class bindings`) için de geçerlidir:

```
<MyComponent :class="{ active: isActive }" />
```

`isActive` değeri doğru (truthy) olduğunda, oluşturulan HTML çıktısı şu şekilde olacaktır:

```
<p class="foo bar active">Hi!</p>
```

Bileşeninizin birden fazla kök öğesi (root element) varsa, bu sınıfı hangi öğenin alacağını belirtmeniz gerekir. Bunu `$attrs` bileşen özelliğini kullanarak yapabilirsiniz:

```
<!-- MyComponent template using $attrs -->
<p :class="$attrs.class">Hi!</p>
```

```
<span>This is a child component</span>
```

```
<MyComponent class="baz" />
```

Oluşturulacak çıktı şu şekildedir:

```
<p class="baz">Hi!</p>
<span>This is a child component</span>
```

Satır İçi Stillerin Bağlanması (Binding Inline Styles)

Nesnelere Bağlama (Binding to Objects)

`:style`, JavaScript nesne değerlerine (object values) bağlamayı destekler. Bu özellik, bir HTML öğesinin `style` özelliğine doğrudan karşılık gelir:

```
data() {
  return {
    activeColor: 'red',
    fontSize: 30
  }
}
```

```
<div :style="{ color: activeColor, fontSize: fontSize + 'px' }"></div>
```

`camelCase` anahtarları önerilse de, `:style` aynı zamanda (gerçek CSS'de nasıl kullanıldıklarına karşılık gelecek şekilde) kebab-case formatındaki CSS özellik anahtarlarını da destekler. Örneğin:

```
<div :style="{ 'font-size': fontSize + 'px' }"></div>
```

Şablonun daha temiz olması için genellikle doğrudan bir stil nesnesine (style object) bağlanmak iyi bir fikirdir:

```
data() {
  return {
```

```
    styleObject: {
      color: 'red',
      fontSize: '13px'
    }
  }
}
```

```
<div :style="styleObject"></div>
```

Yine benzer şekilde, nesne stili bağlaması (object style binding) çoğunlukla nesne döndüren hesaplanmış özelliklerle (`computed properties`) birlikte kullanılır.

`:style` direktifleri, tıpkı `:class` gibi, normal stil nitelikleriyle (style attributes) birlikte de yer alabilir.

Şablon (Template):

```
<h1 style="color: red" :style="'font-size: 1em'">hello</h1>
```

Oluşturulacak çıktı şu şekildedir:

```
<h1 style="color: red; font-size: 1em;">hello</h1>
```

Dizilere Bağlama (Binding to Arrays)

`:style` direktifini birden fazla stil nesnesinden oluşan bir diziye bağlayabiliriz. Bu nesneler birleştirilerek aynı öğeye uygulanacaktır:

```
<div :style="[baseStyles, overridingStyles]"></div>
```

Otomatik Önekleme (Auto-prefixing)

`:style` içinde tarayıcı öneki (`vendor prefix`) gerektiren bir CSS özelliği kullandığınızda, Vue uygun öneki otomatik olarak ekleyecektir. Vue bunu, mevcut tarayıcının hangi stil özelliklerini desteklediğini çalışma zamanında (runtime) kontrol ederek yapar. Tarayıcı belirli bir özelliği desteklemiyorsa, desteklenen birini bulmak amacıyla farklı önekli varyantlar test edilir.

Çoklu Değerler (Multiple Values)

Bir stil özelliğine birden fazla (önekli) değer içeren bir dizi sağlayabilirsiniz. Örneğin:

```
<div :style="{ display: ['-webkit-box', '-ms-flexbox', 'flex'] }"></div>
```

Bu, tarayıcının desteklediği dizideki yalnızca son değeri oluşturacaktır. Bu örnekte, öneksiz flexbox sürümünü destekleyen tarayıcılar için `display: flex` çıktısını verecektir.

Olay Yönetimi (Event Handling)

Olayları Dinlemek (Listening to Events)

DOM olaylarını dinlemek ve tetiklendiklerinde bazı JavaScript kodlarını çalıştırmak için, genellikle `@` simgesiyle kısalttığımız `v-on` direktifini kullanabiliriz. Kullanımı `v-on:click="handler"` şeklinde veya kısayoluyla `@click="handler"` olmalıdır.

İşleyici (`handler`) değeri aşağıdakilerden biri olabilir:

1. **Satır İçi İşleyiciler (Inline handlers):** Olay tetiklendiğinde çalıştırılacak satır içi JavaScript (yerel `onclick` özniteliğine benzer).
2. **Yöntem İşleyicileri (Method handlers):** Bileşen üzerinde tanımlanan bir yönteme (`method`) işaret eden bir özellik adı veya yolu.

Satır İçi İşleyiciler (Inline Handlers)

Satır içi işleyiciler (inline handlers) genellikle basit durumlarda kullanılır. Örneğin:

```
data() {
  return {
    count: 0
  }
}
```

```
<button @click="count++">Add 1</button>
<p>Count is: {{ count }}</p>
```

Metot İşleyicileri (Method Handlers)

Ancak birçok olay işleyicisinin mantığı daha karmaşık olacaktır ve muhtemelen satır içi işleyicilerle (inline handlers) sürdürülebilir/uygulanabilir değildir. Bu nedenle `v-on`, çağırmak istediğiniz bir bileşen yönteminin adını veya yolunu da kabul edebilir.

Örneğin:

```
data() {
  return {
    name: 'Vue.js'
  }
},
methods: {
  greet(event) {
    // `this` inside methods points to the current active instance
    alert(`Hello ${this.name}!`)
    // `event` is the native DOM event
    if (event) {
      alert(event.target.tagName)
    }
  }
}
```

```
<!-- `greet` is the name of the method defined above -->
<button @click="greet">Greet</button>
```

Bir yöntem işleyicisi (method handler), onu tetikleyen yerel DOM `Event` nesnesini otomatik olarak alır; yukarıdaki örnekte, olayı tetikleyen öğeye `event.target` üzerinden erişebiliyoruz.

Metot ve Satır İçi Algılama (Method vs. Inline Detection)

Şablon derleyicisi (`template compiler`), `v-on` değer dizesinin geçerli bir JavaScript tanımlayıcısı (identifier) veya özellik erişim yolu (property access path) olup olmadığını kontrol ederek yöntem işleyicilerini algılar. Örneğin; `foo`, `foo.bar` ve `foo['bar']` yöntem işleyicileri olarak değerlendirilirken, `foo()` ve `count++` satır içi işleyiciler (`inline handlers`) olarak ele alınır.

Satır İçi İşleyicilerde Metot Çağırma (Calling Methods in Inline Handlers)

Doğrudan bir yöntem adına bağlamak yerine, bir satır içi işleyici (inline handler) içinde de yöntemleri çağırabiliriz. Bu, yerel olay yerine yönteme özel argümanlar geçirmemizi sağlar:

```
methods: {
  say(message) {
    alert(message)
  }
}
```

```
<button @click="say('hello')">Say hello</button>
<button @click="say('bye')">Say bye</button>
```

Satır İçi İşleyicilerde Olay Argümanına Erişme (Accessing Event Argument in Inline Handlers)

Bazen bir satır içi işleyicide (inline handler) orijinal DOM olayına da erişmemiz gerekir. Bunu, özel `$event` değişkenini kullanarak bir yönteme iletebilir veya satır içi bir ok fonksiyonu (`arrow function`) kullanabilirsiniz:

```
<!-- using $event special variable -->
<button @click="warn('Form cannot be submitted yet.', $event)">
```

```
  Submit
</button>

<!-- using inline arrow function -->
<button @click="(event) => warn('Form cannot be submitted yet.', event)">
  Submit
</button>
```

```
methods: {
  warn(message, event) {
    // now we have access to the native event
    if (event) {
      event.preventDefault()
    }
    alert(message)
  }
}
```

Olay Değiştiricileri (Event Modifiers)

Olay işleyicilerinin içinde `event.preventDefault()` veya `event.stopPropagation()` çağrısı yapmak oldukça yaygın bir ihtiyaçtır. Bunu yöntemlerin içinde kolayca yapabilsek de, yöntemlerin DOM olay detaylarıyla uğraşmak yerine tamamen veri mantığına odaklanabilmesi çok daha iyi olacaktır.

Bu sorunu çözmek için Vue, `v-on` için **olay niteleyicileri (event modifiers)** sunar. Niteleyicilerin, bir noktayla (.) ayrılan direktif son ekleri olduğunu hatırlayın:

- `.stop`
- `.prevent`
- `.self`
- `.capture`
- `.once`
- `.passive`

```
<!-- the click event's propagation will be stopped -->
<a @click.stop="doThis"></a>
```

```
<!-- the submit event will no longer reload the page -->
<form @submit.prevent="onSubmit"></form>

<!-- modifiers can be chained -->
<a @click.stop.prevent="doThat"></a>

<!-- just the modifier -->
<form @submit.prevent></form>

<!-- only trigger handler if event.target is the element itself -->
<!-- i.e. not from a child element -->
<div @click.self="doThat">...</div>
```

İPUCU (TIP)

`.passive` ve `.prevent` niteleyicilerini birlikte kullanmayın; çünkü `.passive`, tarayıcıya zaten olayın varsayılan davranışını engelleme niyetinde olmadığını bildirir ve bunu yaparsanız muhtemelen tarayıcıdan bir uyarı alırsınız.

Tuş Değiştiriciler (Key Modifiers)

Klavye olaylarını dinlerken genellikle belirli tuşları kontrol etmemiz gerekir. Vue, klavye olaylarını dinlerken `v-on` veya `@` için tuş niteleyicileri (key modifiers) eklememize izin verir:

```
<!-- only call `submit` when the `key` is `Enter` -->
<input @keyup.enter="submit" />
```

`KeyboardEvent.key` aracılığıyla sunulan herhangi bir geçerli tuş adını, kebab-case formatına dönüştürerek doğrudan niteleyici (modifier) olarak kullanabilirsiniz.

```
<input @keyup.page-down="onPageDown" />
```

Yukarıdaki örnekte işleyici (`handler`), yalnızca `$event.key` değeri `'PageDown'` değerine eşit olduğunda çağrılacaktır.

Tuş Takma Adları (Key Aliases)

Vue, en sık kullanılan tuşlar için takma adlar (aliases) sağlar:

- `.enter`
- `.tab`
- `.delete` ("Delete" ve "Backspace" tuşlarının her ikisini de yakalar)
- `.esc`
- `.space`
- `.up`
- `.down`
- `.left`
- `.right`

Sistem Değiştirici Tuşlar (System Modifier Keys)

İlgili niteleyici tuş yalnızca basılı olduğunda fare veya klavye olayı dinleyicilerini tetiklemek için aşağıdaki niteleyicileri kullanabilirsiniz:

- `.ctrl`
- `.alt`
- `.shift`
- `.meta`

NOT

Macintosh klavyelerinde `meta`, Komut (Command - ⌘) tuşudur. Windows klavyelerinde `meta`, Windows tuşudur (⊞). Sun Microsystems klavyelerinde `meta`, içi dolu bir elmas (◆) simgesiyle belirtilir. MIT, Lisp makinesi klavyeleri ve bunların devamı niteliğindeki Knight veya space-cadet gibi özel klavyelerde `meta`, "META" olarak etiketlenmiştir. Symbolics klavyelerinde ise `meta`, "META" veya "Meta" şeklinde etiketlenir.

Örneğin:

```
<!-- Alt + Enter -->
<input @keyup.alt.enter="clear" />

<!-- Ctrl + Click -->
<div @click.ctrl="doSomething">Do something</div>
```

İPUCU (TIP)

Niteleyici tuşların (modifier keys) normal tuşlardan farklı olduğunu unutmayın. `keyup` olaylarıyla birlikte kullanıldıklarında, olay gerçekleştiği anda bu tuşların basılı tutuluyor olması gerekir. Bir başka deyişle, `keyup.ctrl` yalnızca `Ctrl` tuşuna basılı tutarken başka bir tuşu bıraktığınızda tetiklenir; sadece `Ctrl` tuşunu bıraktığınızda tetiklenmez.

`.exact` Değiştiricisi (`.exact` Modifier)

`.exact` niteleyicisi, bir olayı tetiklemek için gereken sistem niteleyicilerinin tam kombinasyonunu kontrol etmenizi sağlar.

```
<!-- this will fire even if Alt or Shift is also pressed -->
<button @click.ctrl="onClick">A</button>

<!-- this will only fire when Ctrl and no other keys are pressed -->
<button @click.ctrl.exact="onCtrlClick">A</button>

<!-- this will only fire when no system modifiers are pressed -->
<button @click.exact="onClick">A</button>
```

Fare Düğmesi Değiştiricileri (Mouse Button Modifiers)

- `.left`
- `.right`
- `.middle`

Bu niteleyiciler, işleyiciyi yalnızca belirli bir fare düğmesi tarafından tetiklenen olaylarla sınırlandırır.

NOT

Bununla birlikte, `.left`, `.right` ve `.middle` niteleyici adlarının tipik sağ el kullanımı için tasarlanmış fare düzenini temel aldığını, ancak gerçek fiziksel düğmeleri değil; sırasıyla "ana (main)", "ikincil (secondary)" ve "yardımcı (auxiliary)" işaretçi aygıtı olay tetikleyicilerini temsil ettiğini unutmayın. Dolayısıyla, sol el kullanımına göre ayarlanmış

bir fare düzeninde "ana" düğme fiziksel olarak sağdaki düğme olabilir, ancak yine de `.left` niteleyici işleyicisini tetikler. Benzer şekilde, bir izleme dörtgeni (`trackpad`), tek parmak dokunuşuyla `.left` işleyicisini, iki parmak dokunuşuyla `.right` işleyicisini ve üç parmak dokunuşuyla `.middle` işleyicisini tetikleyebilir. Aynı şekilde, "fare" olayları üreten diğer aygıtlar ve olay kaynakları da "sol" ve "sağ" kavramlarıyla hiçbir ilgisi olmayan tetikleme modlarına sahip olabilir.

Form Giriş Bağlantıları (Form Input Bindings)

Ön yüzde formlarla çalışırken, form girdi öğelerinin durumunu (state) JavaScript'teki ilgili durumla senkronize etmemiz gerekebilir. Değer bağlamalarını (value bindings) ve değişiklik olay dinleyicilerini (change event listeners) manuel olarak birbirine bağlamak zahmetli olabilir:

```
<input
  :value="text"
  @input="event => text = event.target.value">
```

`v-model` direktifi, yukarıdaki işlemleri basitleştirerek şu hale getirmemize yardımcı olur:

```
<input v-model="text">
```

Buna ek olarak `v-model`; farklı türdeki input'larda, `<textarea>` ve `<select>` elementlerinde kullanılabilir. Üzerinde kullanıldığı elemente bağlı olarak otomatik olarak farklı DOM özelliği (property) ve olay (event) çiftlerine dönüşür:

- Metin türündeki `<input>` elementleri ve `<textarea>` elementleri, `value` özelliğini (property) ve `input` olayını (event) kullanır.
- `<input type="checkbox">` ve `<input type="radio">` elementleri, `checked` özelliğini ve `change` olayını kullanır.
- `<select>` elementi, prop olarak `value` değerini ve olay olarak `change` olayını kullanır.

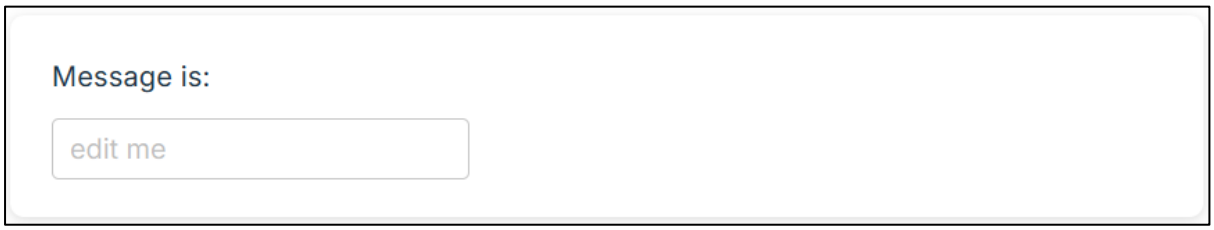
NOT

`v-model`, herhangi bir form elementinde bulunan başlangıçtaki `value`, `checked` veya `selected` özniteliklerini yok sayacaktır. Her zaman bağlı olan mevcut JavaScript durumunu (state) tek gerçek kaynak (source of truth) olarak kabul eder. Başlangıç değerini JavaScript tarafında, `data` seçeneğini kullanarak tanımlamalısınız.

Temel Kullanım (Basic Usage)

Text

```
<p>Message is: {{ message }}</p>
<input v-model="message" placeholder="edit me" />
```



NOT

Çince, Japonca ve Korece gibi IME (Giriş Yöntemi Düzenleyicisi) gerektiren dillerde, IME kompozisyonu sırasında `v-model` değerinin güncellenmediğini fark edeceksiniz. Bu güncellemelere de yanıt vermek istiyorsanız, `v-model` kullanmak yerine kendi input olay dinleyicinizi ve değer bağlamanızı (`value binding`) kullanın.

Çok Satırlı Metin (Multiline Text)

```
<span>Multiline message is:</span>
<p style="white-space: pre-line;">{{ message }}</p>
<textarea v-model="message" placeholder="add multiple lines"></textarea>
```



`<textarea>` içindeki enterpolasyonun (`interpolation`) çalışmayacağını unutmayın. Bunun yerine `v-model` kullanın.

```
<!-- bad -->
<textarea>{{ text }}</textarea>

<!-- good -->
<textarea v-model="text"></textarea>
```

Onay Kutusu (Checkbox)

Tekli onay kutusu (checkbox), boolean (mantıksal) değer:

```
<input type="checkbox" id="checkbox" v-model="checked" />
<label for="checkbox">{{ checked }}</label>
```



Birden fazla onay kutusunu (checkbox) aynı diziye (`array`) veya `Set` değerine de bağlayabiliriz:

```
export default {
  data() {
    return {
      checkedNames: []
    }
  }
}
```

```
<div>Checked names: {{ checkedNames }}</div>

<input type="checkbox" id="jack" value="Jack" v-model="checkedNames" />
<label for="jack">Jack</label>

<input type="checkbox" id="john" value="John" v-model="checkedNames" />
<label for="john">John</label>

<input type="checkbox" id="mike" value="Mike" v-model="checkedNames" />
<label for="mike">Mike</label>
```



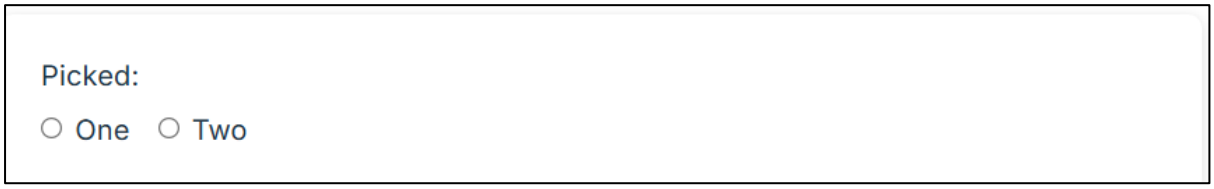
Bu durumda `checkedNames` dizisi, her zaman şu anda işaretli olan kutulardaki değerleri içerecektir.

Radyo Düğmesi (Radio)

```
<div>Picked: {{ picked }}</div>

<input type="radio" id="one" value="One" v-model="picked" />
<label for="one">One</label>

<input type="radio" id="two" value="Two" v-model="picked" />
<label for="two">Two</label>
```

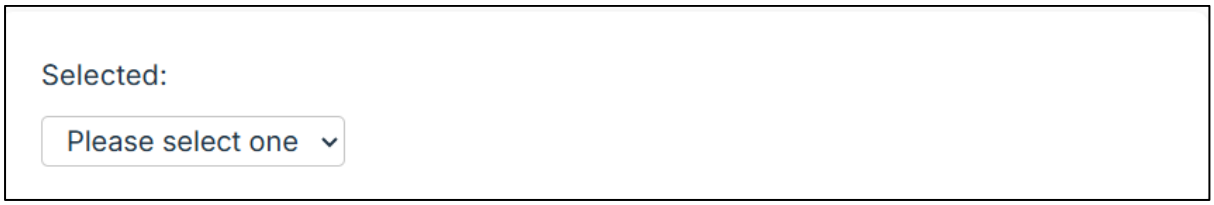


Select

Tekli seçim (`select`):

```
<div>Selected: {{ selected }}</div>

<select v-model="selected">
  <option disabled value="">Please select one</option>
  <option>A</option>
  <option>B</option>
  <option>C</option>
</select>
```



NOT

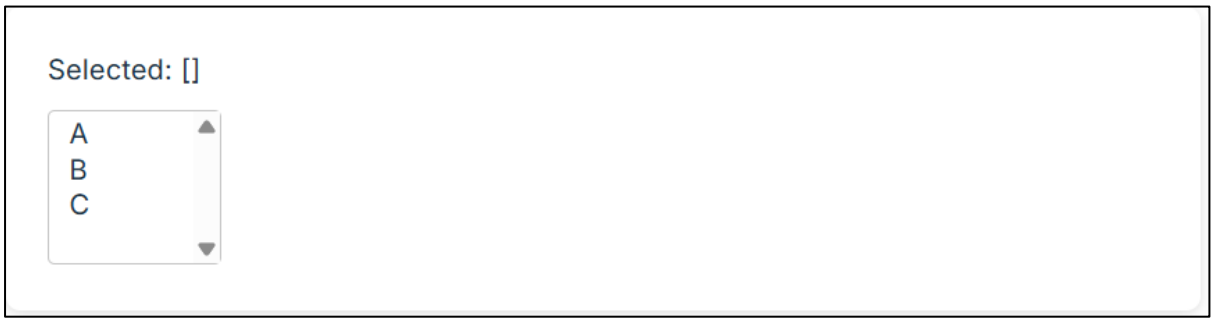
`v-model` ifadesinin başlangıç değeri seçeneklerden hiçbirisiyle eşleşmiyorsa, `<select>` elementi "seçilmemiş" (unselected) durumunda render edilecektir. iOS'te bu durum, kullanıcının ilk öğeyi seçememesine neden olur; çünkü iOS bu senaryoda bir

`change` olayı tetiklemez. Bu nedenle, yukarıdaki örnekte gösterildiği gibi, boş bir değere sahip ve devre dışı bırakılmış (`disabled`) bir seçenek sağlamanız önerilir.

Çoklu seçim (`select`, diziye bağlı):

```
<div>Selected: {{ selected }}</div>

<select v-model="selected" multiple>
  <option>A</option>
  <option>B</option>
  <option>C</option>
</select>
```



Seçenekler (`select options`), `v-for` kullanılarak dinamik olarak render edilebilir:

```
export default {
  data() {
    return {
      selected: 'A',
      options: [
        { text: 'One', value: 'A' },
        { text: 'Two', value: 'B' },
        { text: 'Three', value: 'C' }
      ]
    }
  }
}
```

```
<div>Selected: {{ selected }}</div>

<select v-model="selected">
  <option v-for="option in options" :value="option.value">
    {{ option.text }}
  </option>
```

```
</select>
```

Değer Bağlantıları (Value Bindings)

Radyo butonları (`radio`), onay kutuları (`checkbox`) ve seçim (`select`) seçenekleri için `v-model` bağlama değerleri genellikle statik dizelerdir (veya onay kutusu için mantıksal/boolean değerlerdir):,

```
<!-- `picked` is a string "a" when checked -->
<input type="radio" v-model="picked" value="a" />

<!-- `toggle` is either true or false -->
<input type="checkbox" v-model="toggle" />

<!-- `selected` is a string "abc" when the first option is selected -->
<select v-model="selected">
  <option value="abc">ABC</option>
</select>
```

Ancak bazen değeri, mevcut aktif örnek üzerindeki dinamik bir özelliğe (property) bağlamak isteyebiliriz. Bunu gerçekleştirmek için `v-bind` kullanabiliriz. Buna ek olarak `v-bind` kullanmak, input değerini string (metin) olmayan değerlere bağlamamıza olanak tanır.

Onay Kutusu (Checkbox)

```
<input
  type="checkbox"
  v-model="toggle"
  true-value="yes"
  false-value="no" />
```

`true-value` ve `false-value`, yalnızca `v-model` ile çalışan Vue'ya özgü özniteliklerdir. Burada kutu işaretlendiğinde `toggle` özelliğinin değeri 'yes' olarak, işaretlenmediğinde ise 'no' olarak ayarlanacaktır. Bunları `v-bind` kullanarak dinamik değerlere de bağlayabilirsiniz:

```
<input
  type="checkbox"
  v-model="toggle"
  :true-value="dynamicTrueValue"
  :false-value="dynamicFalseValue" />
```

İPUCU

`true-value` ve `false-value` öznitelikleri, input'un `value` özniteliğini etkilemez; çünkü tarayıcılar form gönderimlerine işaretlenmemiş kutuları dahil etmez. Bir formda iki değerden birinin gönderilmesini garanti etmek için (örneğin "yes" veya "no"), bunun yerine radyo butonları (`radio inputs`) kullanın.

Radyo Düğmesi (Radio)

```
<input type="radio" v-model="pick" :value="first" />
<input type="radio" v-model="pick" :value="second" />
```

`pick`, ilk radyo butonu işaretlendiğinde `first` değerine, ikinci radyo butonu işaretlendiğinde ise `second` değerine ayarlanacaktır.

Liste Seçenekleri (Select Options)

```
<select v-model="selected">
  <!-- inline object literal -->
  <option :value="{ number: 123 }">123</option>
</select>
```

`v-model`, string olmayan değerlerin bağlanmasını da destekler! Yukarıdaki örnekte seçenek seçildiğinde, `selected` değeri `{ number: 123 }` nesne literaline (`object literal`) ayarlanacaktır.

Değiştiriciler (Modifiers)

.lazy

Varsayılan olarak v-model, her input olayından sonra verileri input ile senkronize eder (yukarıda belirtilen IME kompozisyonu istisnası hariç). Bunun yerine change olaylarından sonra senkronizasyon yapmak için lazy niteleyicisini (modifier) ekleyebilirsiniz:

```
<!-- synced after "change" instead of "input" -->
<input v-model.lazy="msg" />
```

.number

Kullanıcı girdisinin otomatik olarak bir sayıya (number) dönüştürülmesini istiyorsanız, v-model tarafından yönetilen input'larınıza number niteleyicisini (modifier) ekleyebilirsiniz:

```
<input v-model.number="age" />
```

Eğer değer parseFloat() ile ayrıştırılamazsa, bunun yerine orijinal (string) değer kullanılır. Özellikle, input boş bırakılırsa (örneğin kullanıcı input alanını temizlediğinde) boş bir string döndürülür. Bu davranış, DOM'daki valueAsNumber özelliğinden farklıdır.

number niteleyicisi, input'un type="number" özniteliğine sahip olması durumunda otomatik olarak uygulanır.

.trim

Kullanıcı girdisindeki boşlukların otomatik olarak kırpılmasını (başındaki ve sonundaki boşlukların temizlenmesini) istiyorsanız, v-model tarafından yönetilen input'larınıza trim niteleyicisini (modifier) ekleyebilirsiniz:

```
<input v-model.trim="msg" />
```

`v-model` with Components

HTML'in yerleşik input türleri her zaman ihtiyaçlarınızı karşılamayabilir. Neyse ki Vue bileşenleri, tamamen özelleştirilmiş davranışlara sahip, yeniden kullanılabilir input'lar oluşturmanıza olanak tanır. Bu input'lar `v-model` ile bile çalışır! Daha fazla bilgi edinmek için Bileşenler kılavuzundaki v-model Kullanımı bölümünü okuyabilirsiniz.

> ***Uyarı:*** *Bu içerik, SCÜ Şarkışla UBYO Web Programlama I dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir. Orijinal dokümantasyon kaynakları (MDN Web Docs, Vue.js, Three.js vb.) kendi orijinal lisanslarına (CC-BY-SA, MIT) tabidir. Bu çalışmanın hiçbir ticari amacı yoktur.*

Kaynakça :

https://vuejs.org/guide/essentials/reactivity-fundamentals.html#reactivity-fundamentals

https://vuejs.org/guide/essentials/computed.html#computed-properties

https://vuejs.org/guide/essentials/class-and-style.html#class-and-style-bindings

https://vuejs.org/guide/essentials/event-handling.html#event-handling

https://vuejs.org/guide/essentials/forms.html#form-input-bindings